INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010
OPERATING SYSTEM

WEEK 14 LECTURE 2

PROCESS ADDRESS SPACE

Process Address Space

LECTURE OUTLINE

- How the process views dynamic memory.
- Basic components of the process address space - "Memory Regions."
- Role played by the Page Fault exception handler in deferring the allocation of page frames to processes
- Illustrate how the kernel creates and deletes whole process address spaces
- APIs and system calls related to address space management.

Process Address Space

- A kernel function gets dynamic memory in a fairly straightforward manner by invoking one of a variety of functions:
  - __get_free_pages( ) or alloc_pages( ) to get pages from the zoned page frame allocator,
  - kmem_cache_alloc( ) or kmalloc( ) to use the slab allocator for specialized or general-purpose objects, and
  - vmalloc( ) or vmalloc_32( ) to get a noncontiguous memory area.
- If the request can be satisfied, each of these functions returns a page descriptor address or a linear address identifying the beginning of the allocated dynamic memory area.
- These simple approaches work for two reasons:
  - The kernel is the highest-priority component of the operating system. If a kernel function makes a request for dynamic memory, it must have a valid reason to issue that request, and there is no point in trying to defer it.
  - The kernel trusts itself. All kernel functions are assumed to be error-free, so the kernel does not need to insert any protection against programming errors.

Process Address Space

- When allocating memory to User Mode processes, the situation is entirely different:
- Process requests for dynamic memory are considered non-urgent.
- When a process's executable file is loaded, for instance, it is unlikely that the process will address all the pages of code in the near future.
- Similarly, when a process invokes malloc() to get additional dynamic memory, it doesn't mean the process will soon access all the additional memory obtained.
- Thus, as a general rule, the kernel tries to defer allocating dynamic memory to User Mode processes.
- Because user programs cannot be trusted, the kernel must be prepared to catch all addressing errors caused by processes in User Mode.
- When a User Mode process asks for dynamic memory, it doesn't get additional page frames; instead, it gets the right to use a new range of linear addresses, which become part of its address space. This interval is called a "memory region."

Process Address Space

❑ Process's Address Space

- The address space of a process consists of all linear addresses that the process is allowed to use.
- Each process sees a different set of linear addresses; the address used by one process bears no relation to the address used by another.
- The kernel may dynamically modify a process address space by adding or removing intervals of linear addresses.
- The kernel represents intervals of linear addresses by means of resources called **memory regions** , which are characterized by an initial linear address, a length, and some access rights.
- For reasons of efficiency, both the initial address and the length of a memory region must be multiples of 4,096, so that the data identified by each memory region completely fills up the page frames allocated to it

Process Address Space

- **Following are some typical situations in which a process gets new memory regions:**

- **When the user types a command at the console, the shell process creates a new process to execute the command. As a result, a fresh address space, and thus a set of memory regions, is assigned to the new process**
- **A running process may decide to load an entirely different program. In this case, the process ID remains unchanged, but the memory regions used before loading the program are released and a new set of memory regions is assigned to the process**
- **A running process may perform a "memory mapping" on a file (or on a portion of it). In such cases, the kernel assigns a new memory region to the process to map the file**

Process Address Space

- **Following are some typical situations in which a process gets new memory regions:**

- **A process may keep adding data on its User Mode stack until all addresses in the memory region that map the stack have been used. In this case, the kernel may decide to expand the size of that memory region**
- **A process may create an IPC-shared memory region to share data with other cooperating processes. In this case, the kernel assigns a new memory region to the process to implement this construct**
- **A process may expand its dynamic area (the heap) through a function such as malloc(). As a result, the kernel may decide to expand the size of the memory region assigned to the heap.**

Process Address Space

Table - System calls related to memory region creation and deletion

System call	Description
brk( )	Changes the heap size of the process
execve( )	Loads a new executable file, thus changing the process address space
_exit( )	Terminates the current process and destroys its address space
fork( )	Creates a new process, and thus a new address space
mmap( ), mmap2( )	Creates a memory mapping for a file, thus enlarging the process address space
mremap( )	Expands or shrinks a memory region
remap_file_pages( )	Creates a non-linear mapping for a file
munmap( )	Destroys a memory mapping for a file, thus contracting the process address space
shmat( )	Attaches a shared memory region
shmdt( )	Detaches a shared memory region

Process Address Space

- It is essential for the kernel to identify the memory regions currently owned by a process (the address space of a process), because that allows the Page Fault exception handler to efficiently distinguish between two types of invalid linear addresses that cause it to be invoked:
  - Those caused by programming errors.
  - Those caused by a missing page; even though the linear address belongs to the process's address space, the page frame corresponding to that address has yet to be allocated.
- The latter addresses are not invalid from the process's point of view; the induced Page Faults are exploited by the kernel to implement demand paging : the kernel provides the missing page frame and lets the process continue.

Memory Descriptor

- All information related to the process address space is included in an object called the memory descriptor of type mm_struct. This object is referenced by the mm field of the process descriptor. The fields of a memory descriptor are listed in Table.

Table - The fields of the memory descriptor

Type	Field	Description
Struct vm_area_struct *	mmap	Pointer to the head of the list of memory region objects
struct rb_root	mm_rb	Pointer to the root of the red-black tree of memory region objects
Struct vm_area_struct *	mmap_cache	Pointer to the last referenced memory region object
unsigned long (*)( )	get_unmapped_area	Method that searches an available linear address interval in the process address space
void (*)( )	unmap_area	Method invoked when releasing a linear address interval
unsigned long	mmap_base	Identifies the linear address of the first allocated anonymous memory region or file memory mapping
unsigned long	free_area_cache	Address from which the kernel will look for a free interval of linear addresses in the process address space

Memory Descriptor

Table - The fields of the memory descriptor

Type	Field	Description
pgd_t *	pgd	Pointer to the Page Global Directory
atomic_t	mm_users	Secondary usage counter
atomic_t	mm_count	Main usage counter
int	map_count	Number of memory regions
Struct rw_semaphore	mmap_sem	Memory regions' read/write semaphore
spinlock_t	page_table_lock	Memory regions' and Page Tables' spin lock
struct list_head	mmlist	Pointers to adjacent elements in the list of memory descriptors
unsigned long	start_code	Initial address of executable code
unsigned long	end_code	Final address of executable code
unsigned long	start_data	Initial address of initialized data
unsigned long	end_data	Final address of initialized data
unsigned long	start_brk	Initial address of the heap
unsigned long	start_stack	Initial address of User Mode stack
unsigned long	arg_start	Initial address of command-line arguments
unsigned long	arg_end	Final address of command-line arguments
unsigned long	env_start	Initial address of environment variables
unsigned long	env_end	Final address of environment variables
unsigned long	rss	Number of page frames allocated to the Process
unsigned long	anon_rss	Number of page frames assigned to anonymous memory mappings

Memory Descriptor

Table - The fields of the memory descriptor

Type	Field	Description
unsigned long	total_vm	Size of the process address space (number of pages)
unsigned long	locked_vm	Number of "locked" pages that cannot be swapped out
unsigned long	shared_vm	Number of pages in shared file memory mappings
unsigned long	exec_vm	Number of pages in executable memory mappings
unsigned long	stack_vm	Number of pages in the User Mode stack
unsigned long	reserved_vm	Number of pages in reserved or special memory regions
unsigned long	def_flags	Default access flags of the memory regions
unsigned long	nr_ptes	Number of Page Tables of this process
unsigned long []	saved_auxv	Used when starting the execution of an ELF program
unsigned int	dumpable	Flag that specifies whether the process can produce a core dump of the memory
cpumask_t	cpu_vm_mask	Bit mask for lazy TLB switches
mm_context_t	context	Pointer to table for architecture-specific information (e.g., LDT's address in 80x86 platforms)

Memory Descriptor

Table - The fields of the memory descriptor

Type	Field	Description
unsigned long	swap_token_time	When this process will become eligible for having the swap token
char	recent_pagein	Flag set if a major Page Fault has recently occurred
int	core_waiters	Number of lightweight processes that are dumping the contents of the process address space to a core file
struct completion *	core_startup_done	Pointer to a completion used when creating a core file
struct completion	core_done	Completion used when creating a core file
rwlock_t	ioctx_list_lock	Lock used to protect the list of asynchronous I/O contexts
struct kioctx *	ioctx_list	List of asynchronous I/O contexts
struct kioctx	default_kioctx	Default asynchronous I/O context
unsigned long	hiwater_rss	Maximum number of page frames ever owned by the process
unsigned long	hiwater_vm	Maximum number of pages ever included in the memory regions of the process

Memory Descriptor

- All memory descriptors are stored in a doubly linked list.
- Each descriptor stores the address of the adjacent list items in the mmlist field.
- The first element of the list is the mmlist field of init_mm, the memory descriptor used by process 0 in the initialization phase.
- The list is protected against concurrent accesses in multiprocessor systems by the mmlist_lock spin lock.
- The mm_users field stores the number of lightweight processes that share the mm_struct data structure
- The mm_count field is the main usage counter of the memory descriptor; all "users" in mm_users count as one unit in mm_count.
- Every time the mm_count field is decreased, the kernel checks whether it becomes zero; if so, the memory descriptor is deallocated because it is no longer in use.

Memory Descriptor

- The mm_alloc( ) function is invoked to get a new memory descriptor.
- Because these descriptors are stored in a slab allocator cache, mm_alloc( ) calls kmem_cache_alloc( ), initializes the new memory descriptor, and sets the mm_count and mm_users field to 1.
- Conversely, the mmput( ) function decreases the mm_users field of a memory descriptor.
- If that field becomes 0, the function releases the Local Descriptor Table, the memory region descriptors, and the Page Tables referenced by the memory descriptor, and then invokes mmdrop( ).
- The latter function decreases mm_count and, if it becomes zero, releases the mm_struct data structure

Memory Descriptor

❑ Memory Descriptor of Kernel Threads

- Kernel threads run only in Kernel Mode, so they never access linear addresses below TASK_SIZE (same as PAGE_OFFSET, usually 0xC0000000).
- Contrary to regular processes, kernel threads do not use memory regions, therefore most of the fields of a memory descriptor are meaningless for them.
- Because the Page Table entries that refer to the linear address above TASK_SIZE should always be identical, it does not really matter what set of Page Tables a kernel thread uses.
- To avoid useless TLB and cache flushes, a kernel thread uses the set of Page Tables of the last previously running regular process.
- To that end, two kinds of memory descriptor pointers are included in every process descriptor: mm and active_mm.

Memory Descriptor

❑ Memory Descriptor of Kernel Threads

- **Two kinds of memory descriptor pointers are included in every process descriptor: mm and active_mm.**

- **The mm field in the process descriptor points to the memory descriptor owned by the process, while the active_mm field points to the memory descriptor used by the process when it is in execution.**

- **For regular processes, the two fields store the same pointer.**

- **Kernel threads, however, do not own any memory descriptor, thus their mm field is always NULL.**

Memory Descriptor

❑ Memory Descriptor of Kernel Threads

- When a kernel thread is selected for execution, its active_mm field is initialized to the value of the active_mm of the previously running process.
- There is, however, a small complication. Whenever a process in Kernel Mode modifies a Page Table entry for a "high" linear address (above TASK_SIZE), it should also update the corresponding entry in the sets of Page Tables of all processes in the system.
- In fact, once set by a process in Kernel Mode, the mapping should be effective for all other processes in Kernel Mode as well.
- Touching the sets of Page Tables of all processes is a costly operation; therefore, Linux adopts a deferred approach.

Memory Descriptor

❑ Memory Descriptor of Kernel Threads

- **In deferred approach in "Noncontiguous Memory Area Management", every time a high linear address has to be remapped (typically by vmalloc( ) or vfree( )), the kernel updates a canonical set of Page Tables rooted at the swapper_pg_dir master kernel Page Global Directory**

- **This Page Global Directory is pointed to by the pgd field of a master memory descriptor , which is stored in the init_mm variable**

- **The swapper process uses init_mm during the initialization phase. However, swapper never uses this memory descriptor once the initialization phase completes.**

Memory Regions

- Linux implements a memory region by means of an object of type vm_area_struct; its fields are shown in Table.

Table - The fields of the memory region object

Type	Field	Description
struct mm_struct *	vm_mm	Pointer to the memory descriptor that owns the region.
unsigned long	vm_start	First linear address inside the region.
unsigned long	vm_end	First linear address after the region.
struct vm_area_struct *	vm_next	Next region in the process list.
pgprot_t	vm_page_prot	Access permissions for the page frames of the region.
unsigned long	vm_flags	Flags of the region.
struct rb_node	vm_rb	Data for the red-black tree
union	shared	Links to the data structures used for reverse mapping.
struct list_head	anon_vma_node	Pointers for the list of anonymous memory regions

Memory Regions

- Linux implements a memory region by means of an object of type vm_area_struct; its fields are shown in Table.

Table - The fields of the memory region object

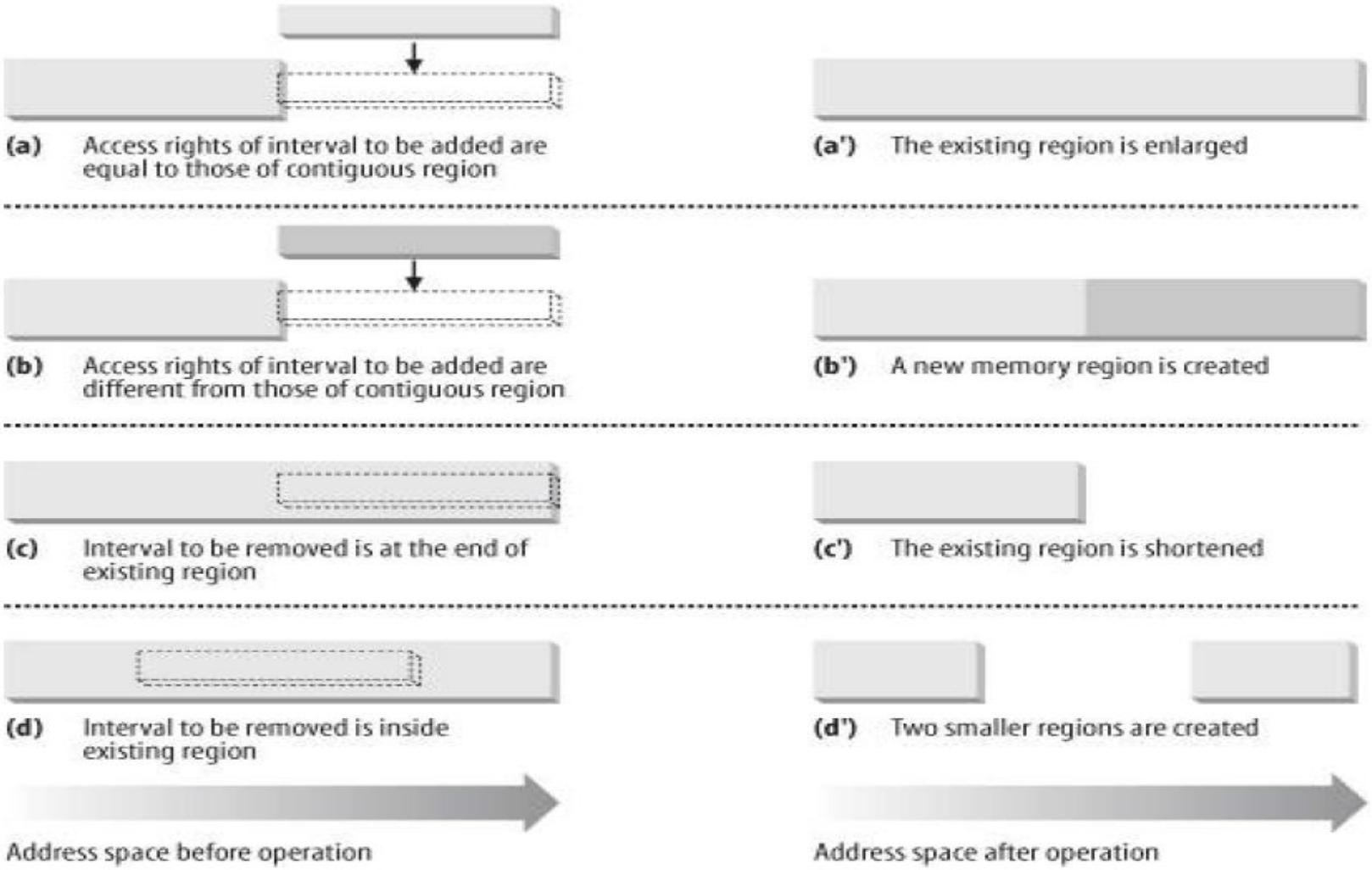
Type	Field	Description
struct anon_vma *	anon_vma	Pointer to the anon_vma data structure
Struct vm_operations_struct*	vm_ops	Pointer to the methods of the memory region.
unsigned long	vm_pgoff	Offset in mapped file For anonymous pages, it is either zero or equal to vm_start/ PAGE_SIZE
struct file *	vm_file	Pointer to the file object of the mapped file, if any.
void *	vm_private_data	Pointer to private data of the memory region.
unsigned long	vm_truncate_count	Used when releasing a linear address interval in a non-linear file memory mapping.

Memory Regions

- Each memory region descriptor identifies a linear address interval.
- The vm_start field contains the first linear address of the interval, while the vm_end field contains the first linear address outside of the interval; vm_end - vm_start thus denotes the length of the memory region.
- The vm_mm field points to the mm_struct memory descriptor of the process that owns the region.
- Memory regions owned by a process never overlap, and the kernel tries to merge regions when a new one is allocated right next to an existing one.
- Two adjacent regions can be merged if their access rights match.

Memory Regions

As shown in Figure, when a new range of linear addresses is added to the process address space, the kernel checks whether an already existing memory region can be enlarged (case a). If not, a new memory region is created (case b). Similarly, if a range of linear addresses is removed from the process address space, the kernel resizes the affected memory regions (case c). In some cases, the resizing forces a memory region to split into two smaller ones (case d) .



Memory Regions

- The vm_ops field points to a vm_operations_struct data structure, which stores the methods of the memory region.

- Only four methods illustrated in Table are applicable to UMA systems.

Table - The methods to act on a memory region

Method	Description
Open	Invoked when the memory region is added to the set of regions owned by a process.
Close	Invoked when the memory region is removed from the set of regions owned by a process.
Nopage	Invoked by the Page Fault exception handler when a process tries to access a page not present in RAM whose linear address belongs to the memory region
Populate	Invoked to set the page table entries corresponding to the linear addresses of the memory region (prefaulting). Mainly used for non-linear file memory mappings.

Memory Regions

❑ Memory Region Data Structures

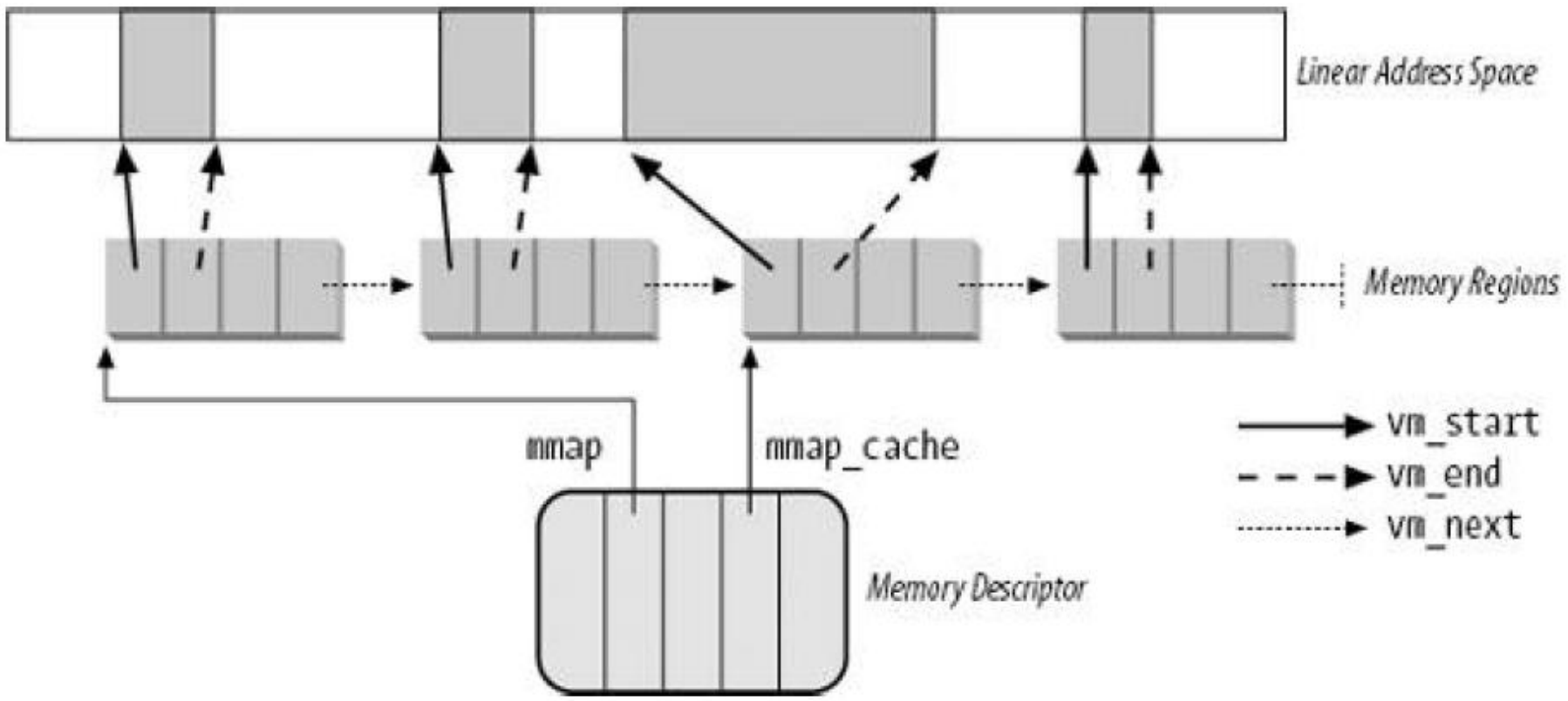
- All the regions owned by a process are linked in a simple list.
- Regions appear in the list in ascending order by memory address; however, successive regions can be separated by an area of unused memory addresses.
- The vm_next field of each vm_area_struct element points to the next element in the list.
- The kernel finds the memory regions through the mmap field of the process memory descriptor, which points to the first memory region descriptor in the list.
- The map_count field of the memory descriptor contains the number of regions owned by the process.
- By default, a process may own up to 65,536 different memory regions; however, the system administrator may change this limit by writing in the /proc/sys/vm/max_map_count file.

Memory Regions

❑ Memory Region Data Structures

- Figure illustrates the relationships among the address space of a process, its memory descriptor, and the list of memory regions.

Figure - Descriptors related to the address space of a process



SOC 3010 OPERATING SYSTEMS
Reading Assignments

Chapter 7 of Text Book

Text Book

- **Understanding the Linux Kernel**
  - **Daniel P/ Bovet and Marco Cesati**
    Edition, O' Reilly Media 2005, ISBN 978-0596005658

PROCESS ADDRESS SPACE

Questions

- State the three functions used by the kernel to get dynamic memory.
- What is meant by a memory region.
- Explain what is meant by address space of a process.
- Indicate some typical situations in which a process gets new memory regions
- Explain the purpose of the following system calls :
  a) brk() b) execve() c) fork() d) mmap()
  e) mremap() f) munmap() g) shmat() h) shmdt()
- What is a Page Fault Exception?
- What are the two main causes for generating page fault exception?
- Explain what is meant by memory descriptor. Why is it required?
- Describe some of the important fields in the memory descriptor.
- Explain the purpose of the following fields in the memory descriptor :
  a) mmap b) mm_rb c) mm_users d) mm_count
  e) map_count f) start_code, end_code g) start_data, end_data
  h) start_brki) start-stack j) arg_start, arg_end k)env_start, env_end
- Write the functions used to get a new memory descriptor and release a memory descriptor.
- What are the two kinds of memory descriptor pointers included in every process descriptor? Why are they required?
- Describe the memory region objects. Explain the various fields in the memory region object data structure.
- Explain the purpose of the following fields in the memory region object :
  a) vm_mm b) vm_start c) vm_end d) vm_next
  f) vm_flags e) vm_page_prot g) vm_rb
- Write the four methods to act on a memory region that are applicable to UMA systems.
- The kernel finds the memory regions through the …….. field of the process memory descriptor, which points to the first memory region descriptor in the list.
- The ……. field of the memory descriptor contains the number of regions owned by the process.
- By default, a process may own up to ……. different memory regions.
  a) 4096 b) 65535 c) 32,768 d) 65,536
- With the help of a schematic, illustrate the relationships among the address space of a process, its memory descriptor, and the list of memory regions.

Memory Regions

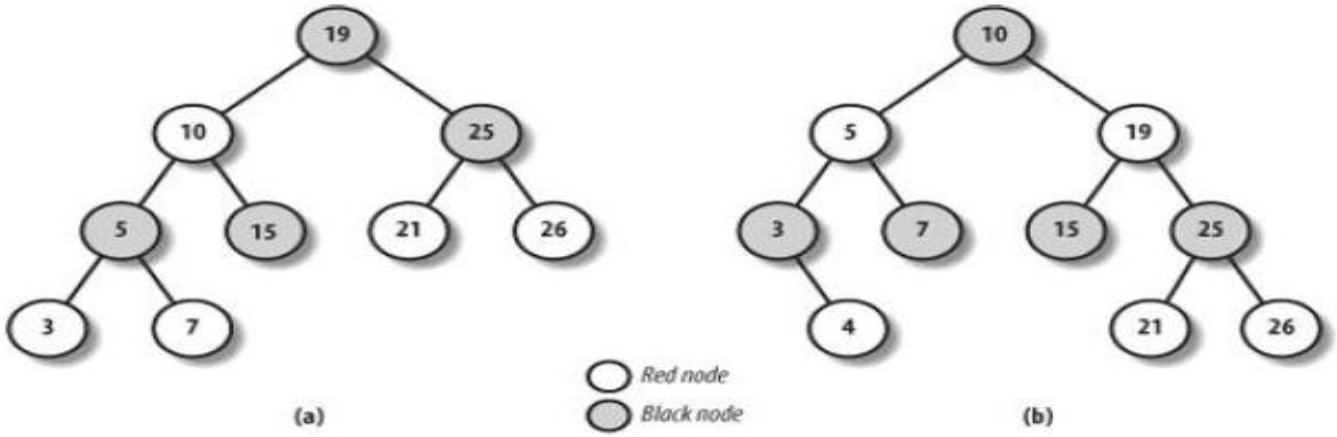
❑ Memory Region Data Structures

- A frequent operation performed by the kernel is to search the memory region that includes a specific linear address. Because the list is sorted, the search can terminate as soon as a memory region that ends after the specific linear address is found.
- However, using the list is convenient only if the process has very few memory regions let's say less than a few tens of them.
- Searching, inserting elements, and deleting elements in the list involve a number of operations whose times are linearly proportional to the list length.
- Although most Linux processes use very few memory regions, there are some large applications, such as object-oriented databases or specialized debuggers for the usage of malloc(), that have many hundreds or even thousands of regions. In such cases, the memory region list management becomes very inefficient, hence the performance of the memory-related system calls degrades to an intolerable point.

Memory Region Data Structures

- Linux stores memory region descriptors in data structures called red-black trees .
- In a red-black tree, each element (or node) usually has two children: a left child and a right child. The elements in the tree are sorted.
- For each node N, all elements of the subtree rooted at the left child of N precede N, while, conversely, all elements of the subtree rooted at the right child of N follow N (see Figure (a); the key of the node is written inside the node itself.
- Moreover, a red-black tree must satisfy four additional rules:
  1. Every node must be either red or black.
  2. The root of the tree must be black.
  3. The children of a red node must be black.
  4. Every path from a node to a descendant leaf must contain the same number of black nodes . When counting the number of black nodes, null pointers are counted as black nodes.
- These four rules ensure that every red-black tree with n internal nodes has a height of at most 2 x log(n + 1).

Figure - Example of red-black trees



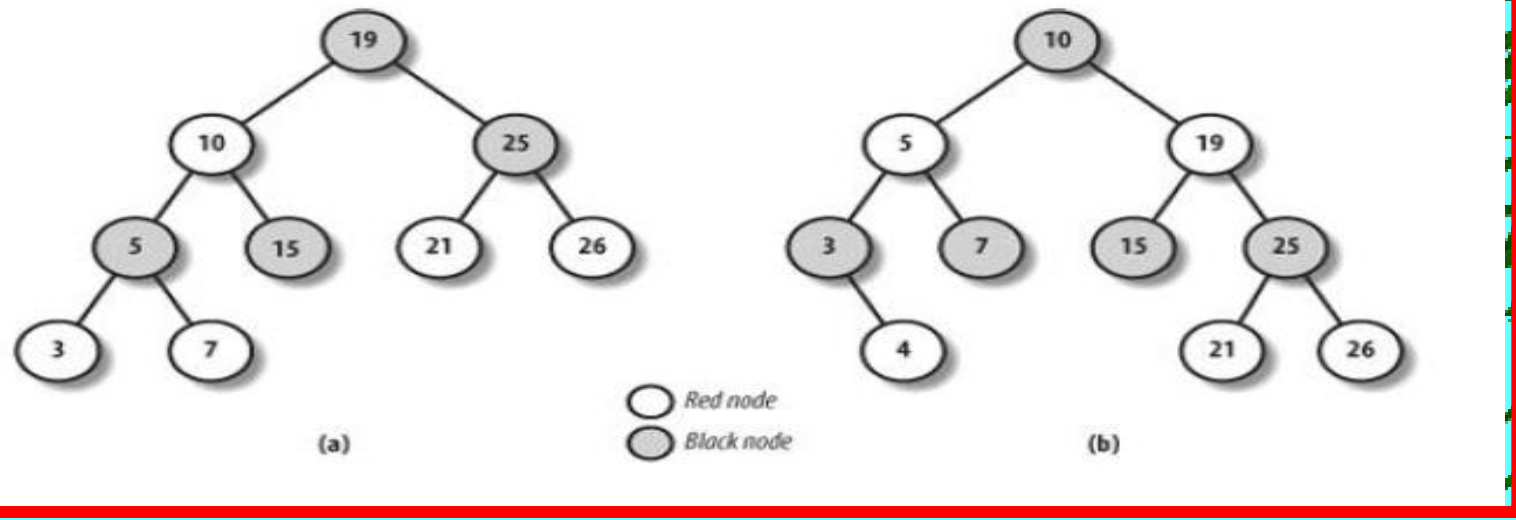
Memory Regions

❑ Memory Region Data Structures

- Searching an element in a red-black tree is thus very efficient, because it requires operations whose execution time is linearly proportional to the logarithm of the tree size.
- In other words, doubling the number of memory regions adds just one more iteration to the operation.
- Inserting and deleting an element in a red-black tree is also efficient, because the algorithm can quickly traverse the tree to locate the position at which the element will be inserted or from which it will be removed.
- Each new node must be inserted as a leaf and colored red.
- If the operation breaks the rules, a few nodes of the tree must be moved or recolored.

Memory Region Data Structures

- For instance, suppose that an element having the value 4 must be inserted in the red-black tree shown in Figure (a).
- Its proper position is the right child of the node that has key 3, but once it is inserted, the red node that has the value 3 has a red child, thus breaking rule 3.
- To satisfy the rule, the color of nodes that have the values 3, 4, and 7 is changed.
- This operation, however, breaks rule 4, thus the algorithm performs a "rotation" on the subtree rooted at the node that has the key 19, producing the new red-black tree shown in Figure (b).



Memory Region Data Structures

- Inserting or deleting an element in a red-black tree requires a small number of operations a number linearly proportional to the logarithm of the tree size.
- Therefore, to store the memory regions of a process, Linux uses both a linked list and a red-black tree.
- Both data structures contain pointers to the same memory region descriptors.
- When inserting or removing a memory region descriptor, the kernel searches the previous and next elements through the red-black tree and uses them to quickly update the list without scanning it.
- The head of the linked list is referenced by the mmap field of the memory descriptor. Each memory region object stores the pointer to the next element of the list in the vm_next field.
- The head of the red-black tree is referenced by the mm_rb field of the memory descriptor.
- Each memory region object stores the color of the node, as well as the pointers to the parent, the left child, and the right child, in the vm_rb field of type rb_node.
- In general, the red-black tree is used to locate a region including a specific address, while the linked list is mostly useful when scanning the whole set of regions.

Memory Region Data Structures

❑ Relation between a page and a memory region

- The term "page" to refer both to a set of linear addresses and to the data contained in this group of addresses.
- In particular, we denote the linear address interval ranging between 0 and 4,095 as page 0, the linear address interval ranging between 4,096 and 8,191 as page 1, and so forth.
- Each memory region therefore consists of a set of pages that have consecutive page numbers.

- There are two kinds of flags associated with a page:
  - ➢A few flags such as Read/Write, Present, or User/Supervisor stored in each Page Table entry
  - ➢A set of flags stored in the flags field of each page descriptor

The first kind of flag is used by the 80 x 86 hardware to check whether the requested kind of addressing can be performed; the second kind is used by Linux for many different purposes

Memory Region Data Structures

- A third kind of flag is ssociated with the pages of a memory region.
- They are stored in the vm_flags field of the vm_area_struct descriptor.
- Some flags offer the kernel information about all the pages of the memory region, such as what they contain and what rights the process has to access each page.
- Other flags describe the region itself, such as how it can grow.

Table - The memory region flags

Flag name	Description
VM_READ	Pages can be read
VM_WRITE	Pages can be written
VM_EXEC	Pages can be executed
VM_SHARED	Pages can be shared by several processes
VM_MAYREAD	VM_READ flag may be set
VM_MAYWRITE	VM_WRITE flag may be set
VM_MAYEXEC	VM_EXEC flag may be set
VM_MAYSHARE	VM_SHARE flag may be set

Memory Region Data Structures

Table - The memory region flags

Flag name	Description
VM_GROWSDOWN	The region can expand toward lower addresses
VM_GROWSUP	The region can expand toward higher addresses
VM_SHM	The region is used for IPC's shared memory
VM_DENYWRITE	The region maps a file that cannot be opened for writing
VM_EXECUTABLE	The region maps an executable file
VM_LOCKED	Pages in the region are locked and cannot be swapped out
VM_IO	The region maps the I/O address space of a device
VM_SEQ_READ	The application accesses the pages sequentially
VM_RAND_READ	The application accesses the pages in a truly random order
VM_DONTCOPY	Do not copy the region when forking a new process
VM_DONTEXPAND	Forbid region expansion through mremap( ) system call
VM_RESERVED	The region is special (for instance, it maps the I/O address space of a device), so its pages must not be swapped out
VM_ACCOUNT	Check whether there is enough free memory for the mapping when creating an IPC shared memory region
VM_HUGETLB	The pages in the region are handled through the extended paging mechanism
VM_NONLINEAR	The region implements a non-linear file mapping

Memory Region Data Structures

- Page access rights included in a memory region descriptor may be combined arbitrarily. It is possible, for instance, to allow the pages of a region to be read but not executed.
- To implement this protection scheme efficiently, the Read, Write, and Execute access rights associated with the pages of a memory region must be duplicated in all the corresponding Page Table entries, so that checks can be directly performed by the Paging Unit circuitry.
- In other words, the page access rights dictate what kinds of access should generate a Page Fault exception.
- The job of figuring out what caused the Page Fault is delegated by Linux to the Page Fault handler, which implements several page-handling strategies.
- The initial values of the Page Table flags (which must be the same for all pages in the memory region,) are stored in the vm_ page_ prot field of the vm_area_struct descriptor.
- When adding a page, the kernel sets the flags in the corresponding Page Table entry according to the value of the vm_ page_ prot field.

Memory Region Data Structures

- Translating the memory region's access rights into the page protection bits is not straightforward for the following reasons:
- In some cases, a page access should generate a Page Fault exception even when its access type is granted by the page access rights specified in the vm_flags field of the corresponding memory region.
- For instance, the kernel may wish to store two identical, writable private pages (whose VM_SHARE flags are cleared) belonging to two different processes in the same page frame; in this case, an exception should be generated when either one of the processes tries to modify the page.
- 80 x 86 processors's Page Tables have just two protection bits, namely the Read/Write and User/Supervisor flags.
- Moreover, the User/Supervisor flag of every page included in a memory region must always be set, because the page must always be accessible by User Mode processes.
- Recent Intel Pentium 4 microprocessors with PAE enabled sport a NX (No eXecute) flag in each 64-bit Page Table entry.

Memory Region Data Structures

- If the kernel has been compiled without support for PAE, Linux adopts the following rules, which overcome the hardware limitation of the 80 x 86 microprocessors:
  - The Read access right always implies the Execute access right, and vice versa.
  - The Write access right always implies the Read access right.

❖ Conversely, if the kernel has been compiled with support for PAE and the CPU has the NX flag, Linux adopts different rules:
  - The Execute access right always implies the Read access right. The Write access right always implies the Read access right.
  - Moreover, to correctly defer the allocation of page frames through the "Copy On Write" technique, the page frame is write-protected whenever the corresponding page must not be shared by several processes.

Memory Region Data Structures

- The 16 possible combinations of the Read, Write, Execute, and Share access rights are scaled down according to the following rules:
- If the page has both Write and Share access rights, the Read/Write bit is set.
- If the page has the Read or Execute access right but does not have either the Write or the Share access right, the Read/Write bit is cleared.
- If the NX bit is supported and the page does not have the Execute access right, the NX bit is set.
- If the page does not have any access rights, the Present bit is cleared so that each access generates a Page Fault exception. However, to distinguish this condition from the real page-not-present case, Linux also sets the Page size bit to 1.You might consider this use of the Page size bit to be a dirty trick, because the bit was meant to indicate the real page size. But Linux can get away with the deception because the 80 x 86 chip checks the Page size bit in Page Directory entries, but not in Page Table entries.
- The downscaled protection bits corresponding to each combination of access rights are stored in the 16 elements of the protection_map array.

Memory Region Handling

- **Group of low-level functions that operate on memory region descriptors :**

- They should be considered auxiliary functions that simplify the implementation of do_mmap( ) and do_munmap( ).
- Those two functions, enlarge and shrink the address space of a process, respectively
- Working at a higher level than the functions we consider here, they do not receive a memory region descriptor as their parameter, but rather the initial address, the length, and the access rights of a linear address interval.

  - Finding the closest region to a given address: find_vma( )
  - Finding a region that overlaps a given interval: find_vma_intersection( )
  - Finding a free interval: get_unmapped_area( )
  - Inserting a region in the memory descriptor list: insert_vm_struct()
  - Allocating a Linear Address Interval : do_mmap( ) function

Memory Region Handling

❑ Finding the closest region to a given address: find_vma( )

- The find_vma( ) function acts on two parameters: the address mm of a process memory descriptor and a linear address addr.
- It locates the first memory region whose vm_end field is greater than addr and returns the address of its descriptor; if no such region exists, it returns a NULL pointer.
- Notice that the region selected by find_vma( ) does not necessarily include addr because addr may lie outside of any memory region.
- Each memory descriptor includes an mmap_cache field that stores the descriptor address of the region that was last referenced by the process.
- This additional field is introduced to reduce the time spent in looking for the region that contains a given linear address.
- Locality of address references in programs makes it highly likely that if the last linear address checked belonged to a given region, the next one to be checked belongs to the same region.

Memory Region Handling

- **Finding a region that overlaps a given interval: find_vma_intersection( )**

- **The find_vma_intersection() function finds the first memory region that overlaps a given linear address interval; the mm parameter points to the memory descriptor of the process, while the start_addr and end_addr linear addresses specify the interval.**
- **The function returns a NULL pointer if no such region exists.**
- **To be exact, if find_vma( ) returns a valid address but the memory region found starts after the end of the linear address interval, vma is set to NULL.**

Memory Region Handling

❑ Finding a free interval: get_unmapped_area( )

- The get_unmapped_area( ) function searches the process address space to find an available linear address interval.
- The len parameter specifies the interval length, while a non-null addr parameter specifies the address from which the search must be started.
- If the search is successful, the function returns the initial address of the new interval; otherwise, it returns the error code -ENOMEM.
- If the addr parameter is not NULL, the function checks that the specified address is in the User Mode address space and that it is aligned to a page boundary.
- Next, the function invokes either one of two methods, depending on whether the linear address interval should be used for a file memory mapping or for an anonymous memory mapping.

Memory Region Handling

❑ Finding a free interval: get_unmapped_area( )

- In linear address interval used for a file memory mapping case, the function executes the get_unmapped_area file operation
- In anonymous memory mapping case, the function executes the get_unmapped_area method of the memory descriptor.
- In turn, this method is implemented by either the arch_get_unmapped_area() function, or the arch_get_unmapped_area_topdown() function, according to the memory region layout of the process.
- Every process can have two different layouts for the memory regions allocated through the mmap( ) system call: either they start from the linear address 0x40000000 and grow towards higher addresses, or they start right above the User Mode stack and grow towards lower addresses.

Memory Region Handling

❑ Inserting a region in the memory descriptor list: insert_vm_struct( )

- **insert_vm_struct( ) inserts a vm_area_struct structure in the memory region object list and red-black tree of a memory descriptor.**
- **It uses two parameters: mm, which specifies the address of a process memory descriptor, and vma, which specifies the address of the vm_area_struct object to be inserted.**
- **The vm_start and vm_end fields of the memory region object must have already been initialized.**
- **The function invokes the find_vma_prepare( ) function to look up the position in the red-black tree mm->mm_rb where vma should go.**
- **Then insert_vm_struct( ) invokes the vma_link( ) function, which in turn:**
  1. **Inserts the memory region in the linked list referenced by mm->mmap.**
  2. **Inserts the memory region in the red-black tree mm->mm_rb.**
  3. **If the memory region is anonymous, inserts the region in the list headed at the corresponding anon_vma data structure**
  4. **Increases the mm->map_count counter.**

Memory Region Handling

❑ Inserting a region in the memory descriptor list: insert_vm_struct( )

- If the region contains a memory-mapped file, the vma_link( ) function performs additional tasks.
- The _ _vma_unlink( ) function receives as its parameters a memory descriptor address mm and two memory region object addresses vma and prev.
- Both memory regions should belong to mm, and prev should precede vma in the memory region ordering.
- The function removes vma from the linked list and the red-black tree of the memory descriptor.
- It also updates mm->mmap_cache, which stores the last referenced memory region, if this field points to the memory region just deleted.

Memory Region Handling

❑ Allocating a Linear Address Interval

➢ **Consider how new linear address intervals are allocated.**

- **To do this, the do_mmap( ) function creates and initializes a new memory region for the current process.**

- **However, after a successful allocation, the memory region could be merged with other memory regions defined for the process.**

- **The function uses the following parameters:**
  file and offset
  File object pointer file and file offset offset are used if the new memory region will map a file into memory. Assume that no memory mapping is required and that file and offset are both NULL.
  addr
  This linear address specifies where the search for a free interval must start.
  len
  The length of the linear address interval.

Memory Region Handling

❑ Allocating a Linear Address Interval - do_mmap() function

prot

This parameter specifies the access rights of the pages included in the memory region.

Possible flags are PROT_READ, PROT_WRITE, PROT_EXEC, and PROT_NONE.

The first three flags mean the same things as the VM_READ, VM_WRITE, and VM_EXEC flags.

PROT_NONE indicates that the process has none of those access rights.

flag

This parameter specifies the remaining memory region flags:

MAP_GROWSDOWN, MAP_LOCKED, MAP_DENYWRITE, and MAP_EXECUTABLE

MAP_SHARED and MAP_PRIVATE

The former flag specifies that the pages in the memory region can be shared among several processes; the latter flag has the opposite effect. Both flags refer to the VM_SHARED flag in the vm_area_struct descriptor.

Memory Region Handling

❑ Allocating a Linear Address Interval

flag

MAP_FIXED The initial linear address of the interval must be exactly the one specified in the addr parameter.

MAP_ANONYMOUS
No file is associated with the memory region

MAP_NORESERVE
The function doesn't have to do a preliminary check on the number of free page frames.

MAP_POPULATE The function should pre-allocate the page frames required for the mapping established by the memory region. This flag is significant only for memory regions that map files and for IPC shared memory regions.

MAP_NONBLOCK Significant only when the MAP_POPULATE flag is set: when pre-allocating the page frames, the function must not block.

Memory Region Handling

❑ Releasing a Linear Address Interval

- When the kernel must delete a linear address interval from the address space of the current process, it uses the do_munmap( ) function.
- The parameters are: the address mm of the process's memory descriptor, the starting address start of the interval, and its length len.
- The interval to be deleted does not usually correspond to a memory region; it may be included in one memory region or span two or more regions.

❖ The do_munmap( ) function

- The function goes through two main phases.
- In the first phase, it scans the list of memory regions owned by the process and unlinks all regions included in the linear address interval from the process address space.
- In the second phase, the function updates the process Page Tables and removes the memory regions identified in the first phase.
- The do_munmap( ) function makes use of the split_vma( ) and unmap_region( ) functions

Memory Region Handling

❑ Releasing a Linear Address Interval

- The do_munmap( ) function makes use of the split_vma( ) and unmap_region( ) functions

❖ The split_vma( ) function

- The purpose of the split_vma( ) function is to split a memory region that intersects a linear address interval into two smaller regions, one outside of the interval and the other inside.
- The function receives four parameters: a memory descriptor pointer mm, a memory area descriptor pointer vma that identifies the region to be split, an address addr that specifies the intersection point between the interval and the memory region, and a flag new_below that specifies whether the intersection occurs at the beginning or at the end of the interval.

Page Fault Exception Handler

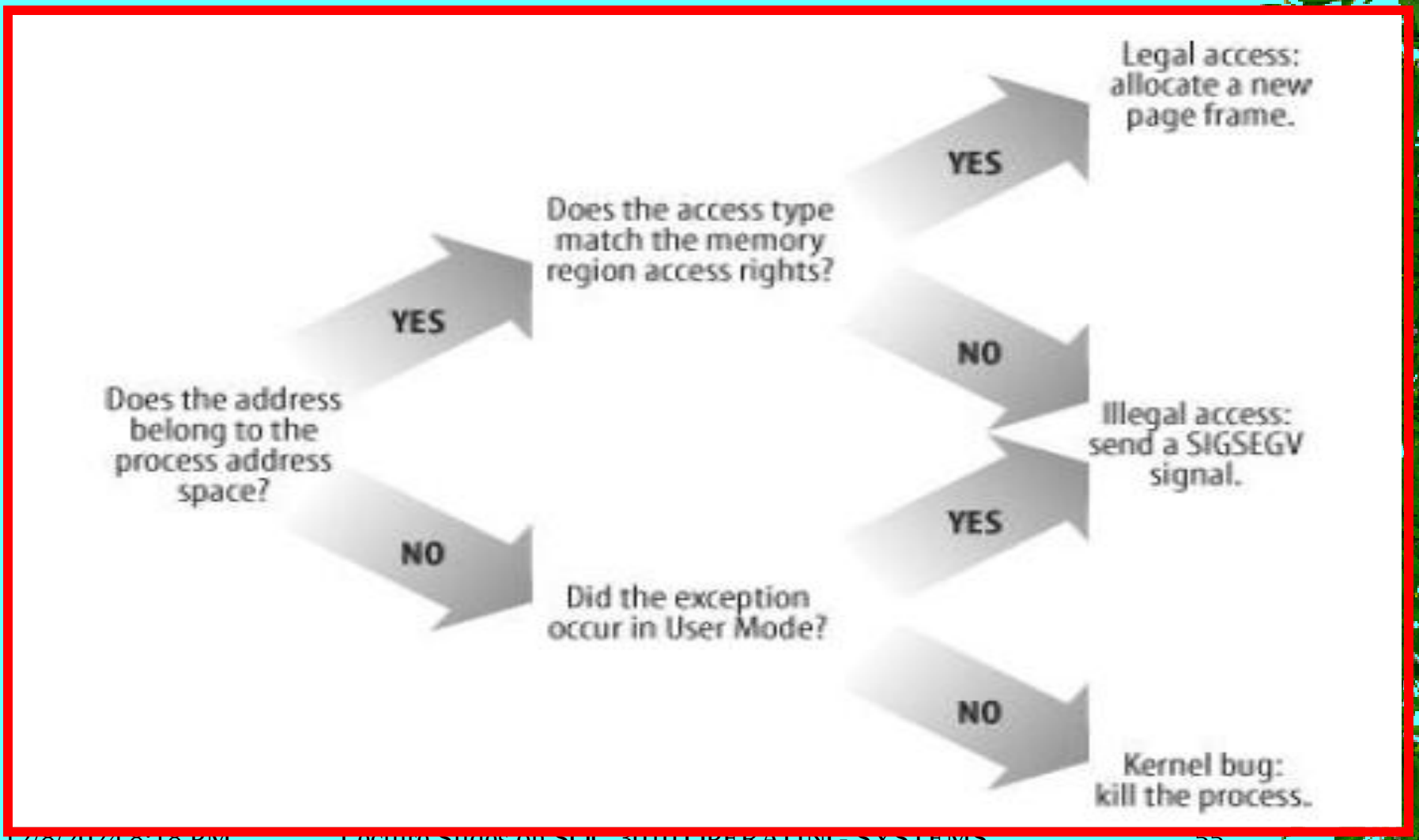
- The Linux Page Fault exception handler must distinguish exceptions caused by programming errors from those caused by a reference to a page that legitimately belongs to the process address space but simply hasn't been allocated yet.

- The memory region descriptors allow the exception handler to perform its job quite efficiently.

- The do_page_fault( ) function, which is the Page Fault interrupt service routine for the 80x86 architecture, compares the linear address that caused the Page Fault against the memory regions of the current process; it can thus determine the proper way to handle the exception according to the scheme that is illustrated in Figure.

Page Fault Exception Handler

- **Overall scheme for the Page Fault handler**



Page Fault Exception Handler

- In practice, things are a lot more complex because the Page Fault handler must recognize several particular subcases that fit awkwardly into the overall scheme, and it must distinguish several kinds of legal access.

- The do_ page_fault() function accepts the following input parameters:

  - The regs address of a pt_regs structure containing the values of the microprocessor registers when the exception occurred.

  - A 3-bit error_code, which is pushed on the stack by the control unit when the exception occurred.

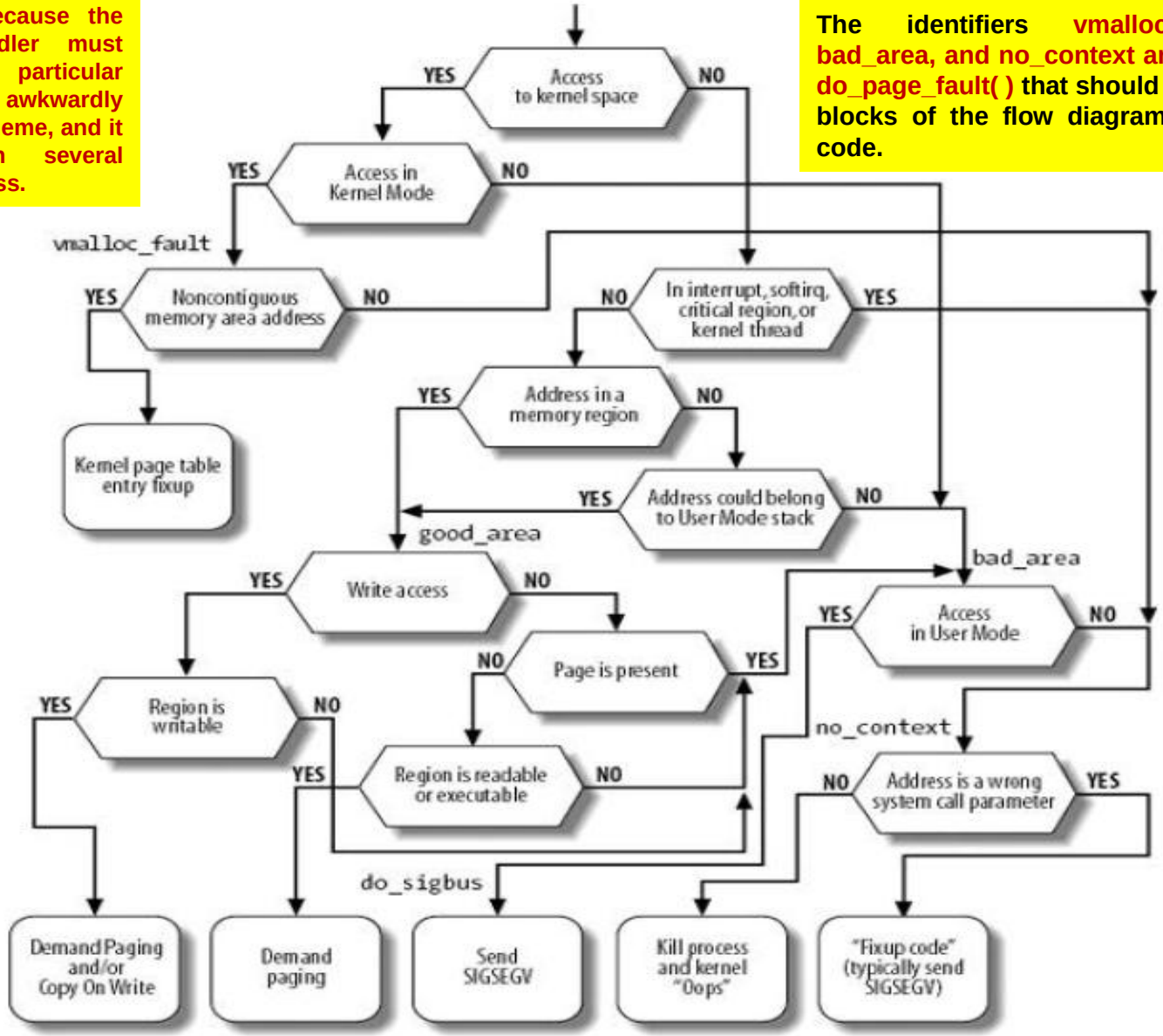
  The bits have the following meanings:
  If bit 0 is clear, the exception was caused by an access to a page that is not present (the Present flag in the Page Table entry is clear); otherwise, if bit 0 is set, the exception was caused by an invalid access right.

Page Fault Exception Handler

- The flow diagram of the Page Fault handler

in practice, things are a lot more complex because the Page Fault handler must recognize several particular subcases that fit awkwardly into the overall scheme, and it must distinguish several kinds of legal access.

The identifiers vmalloc_fault, good_area, bad_area, and no_context are labels appearing in do_page_fault( ) that should help you to relate the blocks of the flow diagram to specific lines of code.



Page Fault Exception Handler

❑ Managing the Heap

- Each Unix process owns a specific memory region called the heap, which is used to satisfy the process's dynamic memory requests.
- The start_brk and brk fields of the memory descriptor delimit the starting and ending addresses, respectively, of that region.
- The following APIs can be used by the process to request and release dynamic memory:

  malloc(size)

  Requests size bytes of dynamic memory; if the allocation succeeds, it returns the linear address of the first memory location.

  calloc(n,size)

  Requests an array consisting of n elements of size size; if the allocation succeeds, it initializes the array components to 0 and returns the linear address of the first element.

  realloc(ptr,size)

  Changes the size of a memory area previously allocated by malloc() or calloc( ) .

Page Fault Exception Handler

❑ Managing the Heap

free(addr)

Releases the memory region allocated by malloc( ) or calloc( ) that has an initial address of addr.

brk(addr)

Modifies the size of the heap directly; the addr parameter specifies the new value of current->mm->brk, and the return value is the new ending address of the memory region (the process must check whether it coincides with the requested addr value).

sbrk(incr)

Is similar to brk( ), except that the incr parameter specifies the increment or decrement of the heap size in bytes.

The brk( ) function differs from the other functions listed because it is the only one implemented as a system call.

All the other functions are implemented in the C library by using brk( ) and mmap( )

The realloc( ) library function can also make use of the mremap( ) system call.

Page Fault Exception Handler

❑ Managing the Heap

- When a process in User Mode invokes the brk( ) system call, the kernel executes the sys_brk(addr) function.
- This function first verifies whether the addr parameter falls inside the memory region that contains the process code; if so, it returns immediately because the heap cannot overlap with memory region containing the process's code
- Because the brk( ) system call acts on a memory region, it allocates and deallocates whole pages. Therefore, the function aligns the value of addr to a multiple of PAGE_SIZE and compares the result with the value of the brk field of the memory descriptor
- If the process asked to shrink the heap, sys_brk( ) invokes the do_munmap( ) function to do the job and then returns

Page Fault Exception Handler

❑ Managing the Heap

- If the process asked to enlarge the heap, sys_brk( ) first checks whether the process is allowed to do so. If the process is trying to allocate memory outside its limit, the function simply returns the original value of mm->brk without allocating more memory
- The function then checks whether the enlarged heap would overlap some other memory region belonging to the process and, if so, returns without doing anything
- If everything is OK, the do_brk( ) function is invoked. If it returns the old brk value, the allocation was successful and sys_brk( ) returns the value addr; otherwise, it returns the old mm->brk value.
- The do_brk( ) function is actually a simplified version of do_mmap( ) that handles only anonymous memory regions.
- do_brk( ) is slightly faster than do_mmap( ), because it avoids several checks on the memory region object fields by assuming that the memory region doesn't map a file on disk.

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 7 of Text Book

Text Book

- **Understanding the Linux Kernel**
  - **Daniel P/ Bovet and Marco Cesati Edition, O' Reilly Media 2005, ISBN 978-0596005658**

PROCESS ADDRESS SPACE

Questions

- Describe the data structure used by Linux to store memory descriptors for implementing efficient search, insertion and deletion.
- Describe the red-black tree data structure used by Linux to store memory descriptors for implementing efficient search, insertion and deletion.
- What are the three kinds of flags associated with a page?
- List the descriptors or entries containing three kinds of flags associated with a page.
- State the purpose of the Read/Write, Present, and User/Supervisor flags in each Page Table entry.
- Discuss memory region flags.
- State the purpose of NX flag in the processor.
- Write the low-level functions to perform the following operations on a memory region descriptor.
  a) Finding the closest region to a given address
  b) Finding a region that overlaps a given interval
  c) Finding a free interval
  d) Inserting a region in the memory descriptor list
  e) Releasing a region from the memory descriptor list
- State the purpose of the following low-level functions in linux and provide the format of each of these functions.
  a) Find_vma() b) find_vma_intersection() c) get_unmapped_area()
  d) insert_vm_struct() e) do_mmap() f) do_munmap() g) vma_link()
  h) vma_unlink() i) split_vma()
- Explain the purpose of do_page_fault() function.
- What are the two input parameters required by the do_page_fault() function? How does the function distinguish between two kinds of page fault exceptions?
- With the help of a schematic, describe the linux Page Fault Exception handler overall scheme.
- Daw the complete flow diagram of the Page Fault Handler and explain all the steps of the Page Fault handler.
- Explain what is meant by demand paging.
- Explain the Copy On Write Approach in Linux
- State the purpose of copy_mm() function.
- Write the system call to create a lightweight process.
- State the purpose of exit_mm() function.
- List and explain all the APIs that can be used by the process to request and release dynamic memory.
- Explain the following APIs that can be used by the process to request and release dynamic memory:
  a) malloc(size) b) calloc(n, size) c) realloc(ptr, size)
  d) free(addr) e) brk(addr) f) sbrk(incr)
- State the purpose of sys_brk(addr) function.

SOC 3010
OPERATING SYSTEM

WEEK 14 LECTURE 2

SYSTEM CALLS & SIGNALS

SOC 3010
OPERATING SYSTEM

SYSTEM CALLS

SYSTEM CALLS

- Operating systems offer processes running in User Mode a set of interfaces to interact with hardware devices such as the CPU, disks, printers, and so on.
- Putting an extra layer between the application and the hardware has several advantages :
- First, it makes programming easier, freeing users from studying low-level programming characteristics of hardware devices.
- Second, it greatly increases system security, since the kernel can check the correctness of the request at the interface level before attempting to satisfy it.
- Last but not least, these interfaces make programs more portable since they can be compiled and executed correctly on any kernel that offers the same set of interfaces.
- Unix systems implement most interfaces between User Mode processes and hardware devices by means of *system calls* issued to the kernel.

POSIX APIs and System Calls

- Question - What is the the difference between an application programmer interface (API) and a system call.
- API is a function definition that specifies how to obtain a given service, while System call is an explicit request to the kernel made via a software interrupt.
- Unix systems include several libraries of functions that provide APIs to programmers.
- Some of the APIs defined by the *libc* standard C library refer to *wrapper routines*, that is, routines whose only purpose is to issue a system call.
- Usually, each system call corresponds to a wrapper routine; the wrapper routine defines the API that application programs should refer to.
- The converse is not true, by the way—an API does not necessarily correspond to a specific system call.
- First of all, the API could offer its services directly in User Mode. (For something abstract like math functions, there may be no reason to make system calls.)
- Second, a single API function could make several system calls.
- Moreover, several API functions could make the same system call but wrap extra functionality around it.
- For instance, in Linux the malloc( ), calloc( ), and free( ) POSIX APIs are implemented in the *libc* library: the code in that library keeps track of the allocation and deallocation requests and uses the brk( ) system call in order to enlarge or shrink the process heap.

POSIX APIs and System Calls

System Call Handler and Service Routines

- When a User Mode process invokes a system call, the CPU switches to Kernel Mode and starts the execution of a kernel function.
- In Linux the system calls must be invoked by executing the int $0x80 Assembly instruction, which raises the programmed exception having vector 128.
- Since the kernel implements many different system calls, the process must pass a parameter called the *system call number* to identify the required system call; the eax register is used for that purpose.
- All system calls return an integer value. The conventions for these return values are different from those for wrapper routines.
- In the kernel, positive or null values denote a successful termination of the system call, while negative values denote an error condition.
- In the latter case, the value is the negation of the error code that must be returned to the application program. The errno variable is not set or used by the kernel.

POSIX APIs and System Calls

System Call Handler and Service Routines

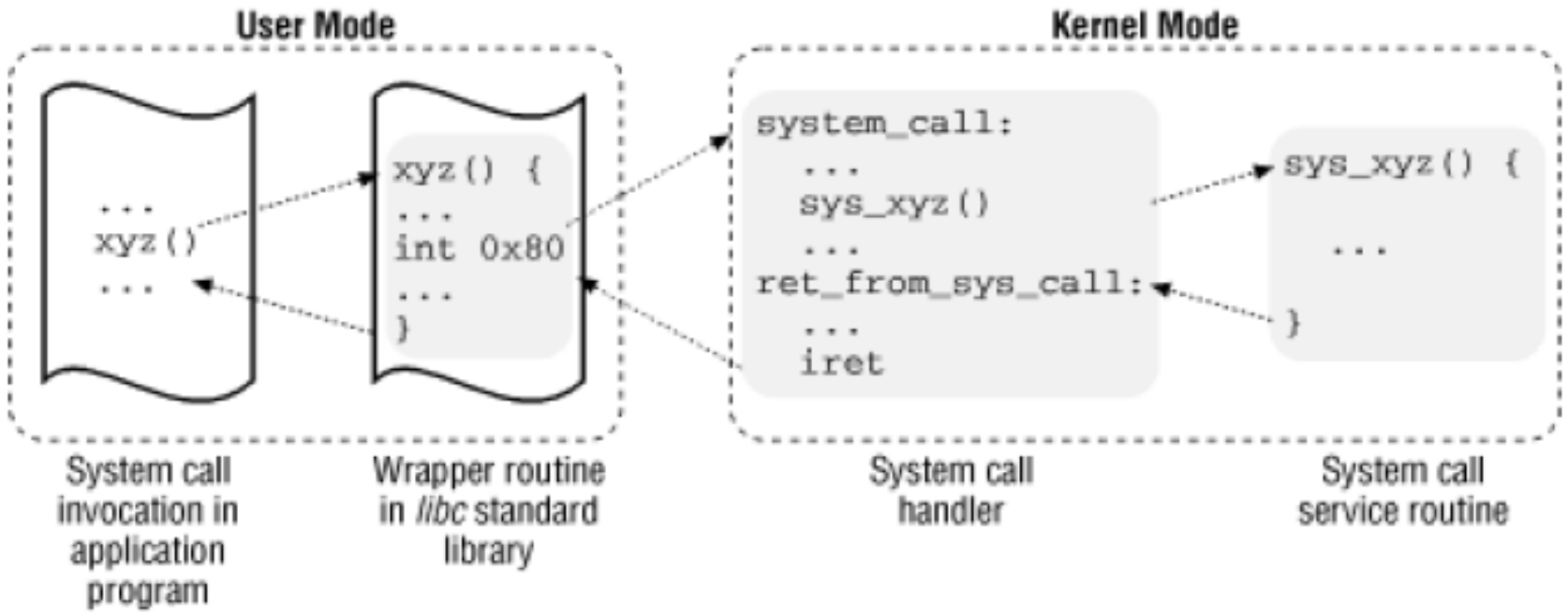
- **The system call handler, which has a structure similar to that of the other exception handlers, performs the following operations:**
  - **Saves the contents of most registers in the Kernel Mode stack (this operation is common to all system calls and is coded in assembly language).**
  - **Handles the system call by invoking a corresponding C function called the *system call service routine*.**
  - **Exits from the handler by means of the ret_from_sys_call( ) function (this function is coded in assembly language).**
- The name of the service routine associated with the *xyz*( ) system call is usually sys_*xyz*(); there are, however, a few exceptions to this rule.

POSIX APIs and System Calls

System Call Handler and Service Routines

- Figure illustrates the relationships among the application program that invokes a system call, the corresponding wrapper routine, the system call handler, and the system call service routine. The arrows denote the execution flow between the functions.

INVOKING A SYSTEM CALL



POSIX APIs and System Calls

System Call Handler and Service Routines

- In order to associate each system call number with its corresponding service routine, the kernel makes use of a *system call dispatch table*; this table is stored in the sys_call_table array and has NR_syscalls entries (usually 256): the *n*th entry contains the service routine address of the system call having number *n*.

- The NR_syscalls macro is just a static limit on the maximum number of implementable system calls: it does not indicate the number of system calls actually implemented.

- Indeed, any entry of the dispatch table may contain the address of the sys_ni_syscall( ) function, which is the service routine of the "nonimplemented" system calls: it just returns the error code -ENOSYS.

Questions

- What are System Calls?
- What are APIs?
- Distinguish between APIs and System Calls.
- What is the relation between **API**s and System Calls?
- Draw the Control Flow Diagram of a System Call showing the relationships among the application program that invokes a system call, the corresponding wrapper routine, the system call handler, and the system call service routine**.**
- What are the operations performed by a System call handler?

SOC 3010
OPERATING SYSTEM

SIGNALS

SIGNALS

Role of Signals

- A signal is a very short message that may be sent to a process or a group of processes.
- The only information given to the process is usually a number identifying the signal; there is no room in standard signals for arguments, a message, or other accompanying information.

SIGNALS

Role of Signals

- A set of macros whose names start with the prefix SIG is used to identify signals
- For instance, the SIGCHLD macro which expands into the value 17 in Linux, yields the identifier of the signal that is sent to a parent process when a child stops or terminates.
- The SIGSEGV macro, which expands into the value 11, yields the identifier of the signal that is sent to a process when it makes an invalid memory reference.

SIGNALS

Signals serve two main purposes:

- To make a process aware that a specific event has occurred
- To cause a process to execute a signal handler function included in its code

SIGNALS IN LINUX

- The first 31 signals listed in Table handled by Linux for the 80x86 architecture (some signal numbers, such those associated with SIGCHLD or SIGSTOP, are architecture-dependent; furthermore, some signals such as SIGSTKFLT are defined only for specific architectures.
- Besides the regular signals described in table, the POSIX standard has introduced a new class of signals denoted as real-time signals ; their signal numbers range from 32 to 64 on Linux.
- They mainly differ from regular signals because they are always queued so that multiple signals sent will be received.
- On the other hand, regular signals of the same kind are not queued: if a regular signal is sent many times in a row, just one of them is delivered to the receiving process.
- Although the Linux kernel does not use real-time signals, it fully supports the POSIX standard by means of several specific system calls.

SIGNALS IN LINUX

Table listing First 31 signals handled by Linux for the 80x86 architecture

#	Signal name	Default action	Comment	POSIX
1.	SIGHUP	Terminate	Hang up controlling terminal or process	Yes
2.	SIGINT	Terminate	Interrupt from keyboard	Yes
3.	SIGQUIT	Dump	Quit from keyboard	Yes
4.	SIGILL	Dump	Illegal instruction	Yes
5.	SIGTRAP	Dump	Breakpoint for debugging	No
6.	SIGABRT	Dump	Abnormal termination	Yes
6.	SIGIOT	Dump	Equivalent to SIGABRT	No
7.	SIGBUS	Dump	Bus error	No
8.	SIGFPE	Dump	Floating-point exception	Yes
9.	SIGKILL	Terminate	Forced-process termination	Yes

SIGNALS IN LINUX

Table listing First 31 signals handled by Linux for the 80x86 architecture

#	Signal name	Default action	Comment	POSIX
10.	SIGUSR1	Terminate	Available to processes	Yes
11.	SIGSEGV	Dump	Invalid memory reference	Yes
12.	SIGUSR2	Terminate	Available to processes	Yes
13.	SIGPIPE	Terminate	Write to pipe with no readers	Yes
14.	SIGALRM	Terminate	Real-timer clock	Yes
15.	SIGTERM	Terminate	Process termination	Yes
16.	SIGSTKFLT	Terminate	Coprocessor stack error	No
17.	SIGCHLD	Ignore	Child process stopped or terminated, or got signal if traced	Yes
18.	SIGCONT	Continue	Resume execution, if stopped	Yes
19.	SIGSTOP	Stop	Stop process execution	Yes

SIGNALS IN LINUX

Table listing First 31 signals handled by Linux for the 80x86 architecture

#	Signal name	Default action	Comment	POSIX
20.	SIGTSTP	Stop	Stop process issued from tty	Yes
21.	SIGTTIN	Stop	Background process requires input	Yes
22.	SIGTTOU	Stop	Background process requires output	Yes
23.	SIGURG	Ignore	Urgent condition on socket	No
24.	SIGXCPU	Dump	CPU time limit exceeded	No
25.	SIGXFSZ	Dump	File size limit exceeded	No
26.	SIGVTALRM	Terminate	Virtual timer clock	No
27.	SIGPROF	Terminate	Profile timer clock	No
28.	SIGWINCH	Ignore	Window resizing	No
29.	SIGIO	Terminate	I/O now possible	No

SIGNALS IN LINUX

Table listing First 31 signals handled by Linux for the 80x86 architecture

#	Signal name	Default action	Comment	POSIX
29	SIGPOLL	Terminate	Equivalent to SIGIO	No
30	SIGPWR	Terminate	Power Supply Failure	No
31	SIGSYS	Dump	Bad system call	No
31	SIGUNUSED	Dump	Equivalent to SIGSYS	No

SIGNALS IN LINUX

- An important characteristic of signals is that they may be sent at any time to a process whose state is usually unpredictable.
- Signals sent to a process that is not currently executing must be saved by the kernel until that process resumes execution.
- Blocking a signal requires that delivery of the signal be held off until it is later unblocked, which exacerbates the problem of signals being raised before they can be delivered.
- Therefore, the kernel distinguishes two different phases related to signal transmission:

  Signal generation

  The kernel updates a data structure of the destination process to represent that a new signal has been sent.

  Signal delivery

  The kernel forces the destination process to react to the signal by changing its execution state, by starting the execution of a specified signal handler, or both.

SIGNALS IN LINUX

- **Each signal generated can be delivered once, at most. Signals are consumable resources: once they have been delivered, all process descriptor information that refers to their previous existence is cancelled.**

- **Signals that have been generated but not yet delivered are called pending signals .**

- **At any time, only one pending signal of a given type may exist for a process; additional pending signals of the same type to the same process are not queued but simply discarded.**

- **Real-time signals are different, though: there can be several pending signals of the same type.**

SIGNALS

In general, a signal may remain pending for an unpredictable amount of time.

The following factors must be taken into consideration:

- **Signals are usually delivered only to the currently running process (that is, to the current process).**
- **Signals of a given type may be selectively blocked by a process.**

  In this case, the process does not receive the signal until it removes the block.
- **When a process executes a signal-handler function, it usually masks the corresponding signal i.e., it automatically blocks the signal until the handler terminates.**
- **A signal handler therefore cannot be interrupted by another occurrence of the handled signal, and the function doesn't need to be reentrant.**

SIGNALS

Although the notion of signals is intuitive, the kernel implementation is rather complex.

The kernel must:

1. **Remember which signals are blocked by each process.**
2. **When switching from Kernel Mode to User Mode, check whether a signal for a process has arrived. This happens at almost every timer interrupt (roughly every millisecond).**

SIGNALS

3. **Determine whether the signal can be ignored. This happens when all of the following conditions are fulfilled:**
   - **The destination process is not traced by another process (the PT_PTRACED flag in the process descriptor ptrace field is equal to 0). If a process receives a signal while it is being traced, the kernel stops the process and notifies the tracing process by sending a SIGCHLD signal to it. The tracing process may, in turn, resume execution of the traced process by means of a SIGCONT signal.**
   - **The signal is not blocked by the destination process.**
   - **The signal is being ignored by the destination process (either because the process explicitly ignored it or because the process did not change the default action of the signal and that action is "ignore").**
4. **Handle the signal, which may require switching the process to a handler function at any point during its execution and restoring the original execution context after the function returns.**

• Moreover, Linux must take into account the different semantics for signals adopted by BSD and System V ; furthermore, it must comply with the rather cumbersome POSIX requirements.

SIGNALS

Actions Performed upon Delivering a Signal

There are three ways in which a process can respond to a signal:

1. Explicitly ignore the signal.

2. Execute the default action associated with the signal. This action, which is predefined by the kernel, depends on the signal type and may be any one of the following:

Terminate	**The process is terminated (killed).**
Dump	**The process is terminated (killed) and a core file containing its execution context is created, if possible; this file may be used for debug purposes.**
Ignore	**The signal is ignored.**
Stop	**The process is stoppedi.e., put in the TASK_STOPPED state, Continue If the process was stopped (TASK_STOPPED), it is put into the TASK_RUNNING state.**

3. Catch the signal by invoking a corresponding signal-handler function.

LINUX IN SIGNALS

- Notice that blocking a signal is different from ignoring it.
- A signal is not delivered as long as it is blocked; it is delivered only after it has been unblocked.
- An ignored signal is always delivered, and there is no further action.
- The SIGKILL and SIGSTOP signals cannot be ignored, caught, or blocked, and their default actions must always be executed.
- Therefore, SIGKILL and SIGSTOP allow a user with appropriate privileges to terminate and to stop, respectively, every process, regardless of the defenses taken by the program it is executing.
- There are two exceptions: it is not possible to send a signal to process 0 (swapper), and signals sent to process 1 (init) are always discarded unless they are caught. Therefore, process 0 never dies, while process 1 dies only when the init program terminates.

LINUX IN SIGNALS

- A signal is fatal for a given process if delivering the signal causes the kernel to kill the process.
- The SIGKILL signal is always fatal; moreover, each signal whose default action is "Terminate" and which is not caught by a process is also fatal for that process.
- Notice, however, that a signal caught by a process and whose corresponding signal-handler function terminates the process is not fatal, because the process chose to terminate itself rather than being killed by the kernel.

SIGNALS

Generating a Signal

- **Many kernel functions generate signals: they accomplish the first phase of signal handling by updating one or more process descriptors as needed.**

- **They do not directly perform the second phase of delivering the signal but, depending on the type of signal and the state of the destination processes, may wake up some processes and force them to receive the signal.**

- **When a signal is sent to a process, either from the kernel or from another process, the kernel generates it by invoking one of the functions listed in Table**

SIGNALS

Delivering a Signal

- We assume that the kernel noticed the arrival of a signal and invoked one of the functions to prepare the process descriptor of the process that is supposed to receive the signal.
- But in case that process was not running on the CPU at that moment, the kernel deferred the task of delivering the signal.
- We now turn to the activities that the kernel performs to ensure that pending signals of a process are handled.
- The kernel checks the value of the TIF_SIGPENDING flag of the process before allowing the process to resume its execution in User Mode.
- Thus, the kernel checks for the existence of pending signals every time it finishes handling an interrupt or an exception.

SIGNALS

Catching the Signal

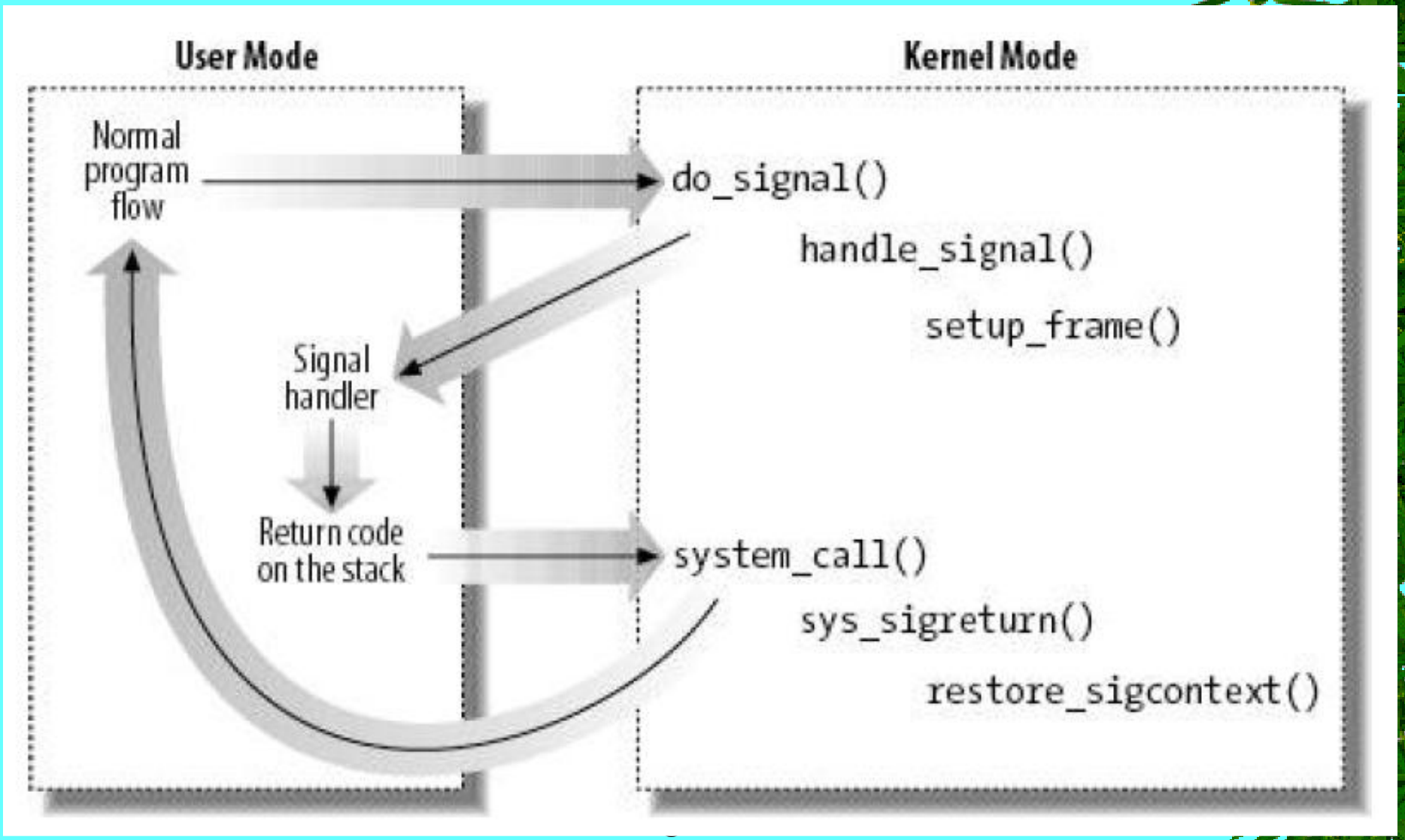
- Signal handlers are functions defined by User Mode processes and included in the User Mode code segment.
- The handle_signal( ) function runs in Kernel Mode while signal handlers run in User Mode; this means that the current process must first execute the signal handler in User Mode before being allowed to resume its "normal" execution.
- Moreover, when the kernel attempts to resume the normal execution of the process, the Kernel Mode stack no longer contains the hardware context of the interrupted program, because the Kernel Mode stack is emptied at every transition from User Mode to Kernel Mode.

❑Catching the Signal

- Figure illustrates the flow of execution of the functions involved in catching a signal.
- A nonblocked signal is sent to a process.
- When an interrupt or exception occurs, the process switches into Kernel Mode.
- Right before returning to User Mode, the kernel executes the do_signal() function, which in turn handles the signal (by invoking handle_signal()) and sets up the User Mode stack (by invoking setup_frame() or setup_rt_frame()).
- When the process switches again to User Mode, it starts executing the signal handler, because the handler's starting address was forced into the program counter.
- When that function terminates, the return code placed on the User Mode stack by the setup_frame( ) or setup_rt_frame( ) function is executed.
- This code invokes the sigreturn( ) or the rt_sigreturn( ) system call; the corresponding service routines copy the hardware context of the normal program to the Kernel Mode stack and restore the User Mode stack back to its original state (by invoking restore_sigcontext( )).
- When the system call terminates, the normal program can thus resume its execution.

SIGNALS

- **Catching the Signal**



SIGNALS

❑Some of the most significant system calls

kill( ) System Call

The kill(pid,sig) system call is commonly used to send signals to conventional processes or multithreaded applications; its corresponding service routine is the sys_kill( ) function.

The integer pid parameter has several meanings, depending on its numerical value:

1. pid > 0 — The sig signal is sent to the thread group of the process whose PID is equal to pid.
2. pid = 0 — The sig signal is sent to all thread groups of the processes in the same process group as the calling process.
3. pid = -1 — The signal is sent to all processes, except swapper (PID 0), init (PID 1), and current.
4. pid < -1 — The signal is sent to all thread groups of the processes in the process group -pid.

SIGNALS

- Some of the most significant system calls

tkill( ) and tgkill( ) System Calls

- The tkill( ) and tgkill( ) system calls send a signal to a specific process in a thread group.
- The pthread_kill( ) function of every POSIX-compliant pthread library invokes either of them to send a signal to a specific lightweight process.
- The tkill( ) system call expects two parameters: the PID pid of the process to be signaled and the signal number sig.
- The sys_tkill( ) service routine fills a siginfo table, gets the process descriptor address, makes some permission checks and invokes specific_send_sig_info( ) to send the signal.
- The tgkill( ) system call differs from tkill( ) because it has a third parameter: the thread group ID (tgid) of the thread group that includes the process to be signaled.

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 9 of Text Book

Text Book

- **Understanding the Linux Kernel**
  - **Daniel P/ Bovet and Marco Cesati Edition, O' Reilly Media 2005, ISBN 978-0596005658**

PROCESS ADDRESS SPACE

Questions

- What is a signal. State the two main purposes served by a signal.
- State the difference between regular (standard) signals and real-time signals.
- The number of regular signals in linux is …… (numbered from … to….) and the number of POSIX real-time signals supported in linux is ……..(numbered from …. to ……).
- The following table lists the names of the signals handled by Linux for the 80x86 architecture. Write the purpose/function associated with each of the signal and default action performed :

#	Signal Name	Purpose	Default Action
1	SIGHUP		
2	SIGINT		
3	SIGQUIT		
4	SIGILL		
5	SIGTRAP		
6	SIGABRT		
7	SIGBUS		
8	SIGFPE		
9	SIGKILL		
11	SIGSEGV		
14	SIGALRM		
15	SIGTERM		
17	SIGCHLD		
18	SIGCONT		
19	SIGSTOP		

- Describe the following two different phases related to signal transmission/reception:
  a) Signal Generation (sending) b) Signal Delivery (Receiving)
- What are pending signals?
- Describe the Actions Performed by a process upon Delivering (Receiving) a Signal.
- What are the three ways in which a process can respond to a signal?
- List all the conditions that can generate Signals?
- Explain what is meant by a fatal signal for a given process.
- Distinguish between blocking and ignoring signals. Indicate the two signals which cannot be ignored, caught, or blocked, and their default actions must always be executed.
- Explain what is meant by catching a signal.
- With the help of a schematic, illustrate the flow of execution of the functions involved in catching a signal.
- Explain the following system calls associated with the signals
  a)kill b) tkill( ) c) tgkill( )

SOC 3010
OPERATING SYSTEM

END OF

WEEK 14 LECTURE 2

PROCESS ADDRESS SPACE, SYSTEM CALLS & SIGNALS

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG